Explainable Student Dropout Risk Prediction Using Machine Learning

A Mini Project Report submitted in fulfillment of the requirements for the Machine Learning Project

BACHELOR OF TECHNOLOGY
IN
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

Submitted by

AMARA BHANU PRASAD (A24126552068)
BODALA HARSHAD RAM (A24126552073)
V LEKHYA SRI (A24126552123)
DAGGUBILLI SOMYA (A24126552258)

Under the guidance of
Dr. Appala Srinuvasu Muttipati
Associate Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A+' Grade)
Sangivalasa, Bheemili Mandal, Visakhapatnam Dist. (A.P)

2026-2027

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

SANGIVALASA, VISAKHAPATNAM

2024-2028

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “EXPLAINABLE STUDENT DROPOUT RISK PREDICTION USING MACHINE LEARNING” submitted by 1. AMARA BHANU PRASAD (Regd No. A24126552068), 2. BODALA HARSHAD RAM (Regd No. A24126552073), 3. V LEKHYA SRI (Regd No. A24126552123), 4. DAGGUBILLI SOMYA (Regd No. A24126552258), in fulfillment of the requirements for the Machine Learning Project in the Department of Computer Science and Engineering (AI&ML) during the year 2026-2027 of Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam, is a record of bonafide work to be carried out under the guidance and supervision named below, pending review and signature.

PROJECT GUIDE:
Dr. Appala Srinuvasu Muttipati
ASSOCIATE PROFESSOR
DEPARTMENT OF CSE (AI&ML)
ANITS

HEAD OF THE DEPARTMENT:
Dr. K. Selvani Deepti
DEPARTMENT OF CSE (AI&ML)
ANITS

DECLARATION

We hereby declare that the project report entitled “EXPLAINABLE STUDENT DROPOUT RISK PREDICTION USING MACHINE LEARNING”, submitted for the Machine Learning Project, is a record of original work carried out by us under the guidance of Dr. Appala Srinuvasu Muttipati, Associate Professor, Department of Computer Science and Engineering (AI&ML). The implementation, experiments, and results reported were produced and verified by us; all external ideas, definitions and results drawn from published sources have been acknowledged in the References section.

We further declare that this report has not been submitted, in full or in part, for the award of any other degree, diploma, or similar title to this or any other institution.

Place: Visakhapatnam
Date: 09-10-2026

AMARA BHANU PRASAD (A24126552068)

BODALA HARSHAD RAM (A24126552073)

V LEKHYA SRI (A24126552123)

DAGGUBILLI SOMYA (A24126552258)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Appala Srinuvasu Muttipati, Associate Professor, Department of CSE (AI&ML), for his valuable guidance, continuous encouragement, and constructive feedback throughout this project. His emphasis on sound experimental design and honest reporting of results shaped the way we approached the problem of early dropout prediction.

We thank Dr. K. Selvani Deepti, Head of the Department, for providing the departmental resources, laboratory access, and computing environment necessary to carry out this work. We are equally grateful to all the faculty members of the Department of Computer Science and Engineering (AI&ML) for their valuable advice and encouragement.

We also acknowledge the creators of the *Predict Students' Dropout and Academic Success* dataset and the UCI Machine Learning Repository for making the data openly available, and the developers of the scikit-learn, imbalanced-learn, XGBoost, SHAP and LIME libraries on which this work is built.

Finally, we thank our classmates for the many useful discussions during the course of this project, and our families for their patience and support throughout the duration of this work.

TEAM MEMBERS:

AMARA BHANU PRASAD (A24126552068)
BODALA HARSHAD RAM (A24126552073)
V LEKHYA SRI (A24126552123)
DAGGUBILLI SOMYA (A24126552258)

ABSTRACT

Student dropout is a major concern for higher-education institutions: it affects students' careers, institutional performance and public investment in education. Identifying at-risk students early enough for tutors and support services to act is therefore a practical machine-learning problem. This project develops and evaluates an explainable early-warning model that classifies each student as **Dropout**, **Enrolled** or **Graduate** using the *Predict Students' Dropout and Academic Success* dataset from the UCI Machine Learning Repository (4,424 students, 36 input attributes).

The prediction point is fixed at the **end of the first semester**, and all six second-semester attributes are removed so that no future information leaks into the model, leaving 30 candidate features. A leakage-safe pipeline (median imputation, standard scaling, random over-sampling applied only to training folds, and mutual-information feature selection retaining 20 features) is combined with five classifiers: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine and XGBoost. Models are compared with stratified 5-fold cross-validation on the training split (80%), using Macro F1 as the primary criterion because the classes are imbalanced.

XGBoost achieved the best cross-validated Macro F1 (0.6941). On the untouched test set of 885 students it reached an accuracy of 71.19%, Macro F1 of 0.6741, balanced accuracy of 0.6865 and a weighted one-vs-rest ROC-AUC of 0.8783. The Enrolled class was the hardest to separate (F1 = 0.495), while Dropout and Graduate were identified with F1 of 0.715 and 0.812. SHAP and LIME explanations agree that the number of curricular units approved in the first semester and whether tuition fees are up to date are the dominant risk signals, followed by age at enrollment, first-semester workload and grades, and debtor status. Finally, a simple probability-based risk-tier scheme is demonstrated: students in the High tier (predicted dropout probability of at least 0.60) were 87.8% actual dropouts, compared with 12.9% in the Low tier.

Keywords: *Student Dropout Prediction, Early-Warning System, Explainable Machine Learning, SHAP, LIME, Class Imbalance, Feature Selection, XGBoost, Educational Data Mining.*

TABLE OF CONTENTS

I Declaration iii

II Acknowledgement iv

III Abstract v

1. Introduction 1

2. Related Work 2

3. Methodology 4

4. Results 8

5. Discussion 18

6. Conclusion 20

References 22

Appendix A: Source Code Implementation 23

Appendix B: Supplementary Code for the Risk-Tier Analysis (Section 4.9) 31

1. Introduction

Higher-education institutions record large amounts of information about their students, from admission details and family background to semester-wise academic results. Yet a substantial fraction of students never complete their degrees. Dropout and academic failure are an obstacle to economic growth, employment and productivity, and they directly affect students, their families and the institutions themselves [1]. Classical theory describes dropout not as a single event but as a longitudinal process shaped by the student's academic and social integration into the institution, which implies that signals of risk are usually visible well before the student actually leaves.

Machine learning can turn those signals into an *early-warning system*: a model that estimates each student's risk and helps tutors prioritise support. Building such a system that is genuinely useful, however, raises several practical challenges that this project addresses explicitly:

- **Timing and information leakage.** A model is only useful as an early warning if it uses information that is available at the moment of prediction. The source dataset contains both first- and second-semester results; using the second-semester columns would silently let the model “see the future”. This project fixes the prediction point at the end of the first semester and removes all second-semester attributes.
- **Class imbalance.** The three outcome classes are unequal (Graduate 49.9%, Dropout 32.1%, Enrolled 17.9%). Plain accuracy can hide poor performance on the minority class [4], so imbalance handling and class-sensitive metrics are essential.
- **Methodological leakage.** Pre-processing, resampling and feature selection must be fitted only on training data; otherwise cross-validation estimates are optimistically biased [3]. All such steps are therefore embedded in a single pipeline that is re-fitted inside every cross-validation fold.
- **Explainability.** Advisers will act on a prediction only if they understand it. Post-hoc explanation methods, namely SHAP [7] and LIME [8], are used to identify which factors drive both the model as a whole and individual predictions.

Feature selection is used to keep the model compact and interpretable, in line with the broader literature on variable and feature selection [2]. The objectives of the project, as set out in the project brief, are:

- to preprocess the data and define a leakage-free early-warning prediction stage;
- to handle class imbalance without contaminating validation or test data;
- to perform feature selection and compare multiple machine-learning classifiers using appropriate multi-class metrics (confusion matrix, F1, ROC-AUC and balanced accuracy);
- to add SHAP and LIME explanations that identify the factors influencing individual predictions and overall risk; and
- to outline an early-warning framework that converts predicted probabilities into actionable risk tiers.

The remainder of this report is organised as follows. Section 2 reviews related work. Section 3 describes the dataset and methodology. Section 4 presents the results. Section 5 discusses the findings and limitations, and Section 6 concludes with recommendations and future work. The complete source code is given in the Appendix.

2. Related Work

2.1 Student dropout and educational data mining

Classical theories of student retention frame withdrawal from higher education as the outcome of a continuing interaction between the student and the academic and social systems of the institution. This longitudinal view motivates early-warning approaches: if disengagement builds over time, measurable indicators in the first semester should already carry predictive information. The dataset used in this project was assembled by Realinho et al. [1] from several disjoint institutional data sources at a Portuguese polytechnic institute. It combines demographic, socio-economic, macroeconomic and academic attributes, and was created to feed a Learning Analytics tool that gives the tutoring team an estimate of each student's risk of dropout and failure. The authors present it explicitly as a three-category classification task with a strong imbalance towards one class.

2.2 Class imbalance

Learning from imbalanced data is a well-studied problem; surveys describe sampling-based, cost-sensitive and algorithm-level remedies and stress that accuracy is a poor guide when classes are unequal [4]. Synthetic Minority Over-sampling Technique (SMOTE) creates new minority examples by interpolating between neighbours. This is awkward for data in which many columns are integer-coded categories (course, application mode, parental occupation), because interpolated values may not correspond to any valid category. Random over-sampling, which simply duplicates minority-class rows, avoids this problem and is available in the imbalanced-learn library, whose pipeline object also guarantees that resampling is applied only to training folds.

2.3 Feature selection and selection bias

Guyon and Elisseeff [2] distinguish filter, wrapper and embedded approaches to variable selection. Filters based on mutual information can capture non-linear dependence between a feature and the class; scikit-learn's *mutual_info_classif* implements a k-nearest-neighbour estimator of this quantity. A critical caution from Ambroise and McLachlan [3] is that selecting features on the full dataset before cross-validation produces strongly optimistic error estimates; selection must be repeated inside each training fold. This project follows that guidance by placing the selector inside the model pipeline.

2.4 Classification algorithms

The comparison covers a linear model (Logistic Regression), a single tree (Decision Tree), bagged trees (Random Forest [5]), a kernel method (Support Vector Machine with an RBF kernel) and gradient-boosted trees (XGBoost [6]). Tree ensembles are generally strong on tabular data with mixed numeric and categorical-coded attributes, which is the situation here.

2.5 Explainable machine learning

Highly accurate models such as ensembles are not directly interpretable. SHAP [7] assigns each feature a Shapley-value contribution to a prediction, providing both local explanations and, by aggregation, global importance; a model-agnostic permutation explainer is used here to approximate these values. LIME [8] explains a single prediction by fitting a simple interpretable surrogate model to perturbed samples around that instance. Post-hoc explanations describe the model rather than the world and are not guaranteed to be faithful; accordingly, explanations in this report are read as statements about *model behaviour and association*, not about causes of dropout.

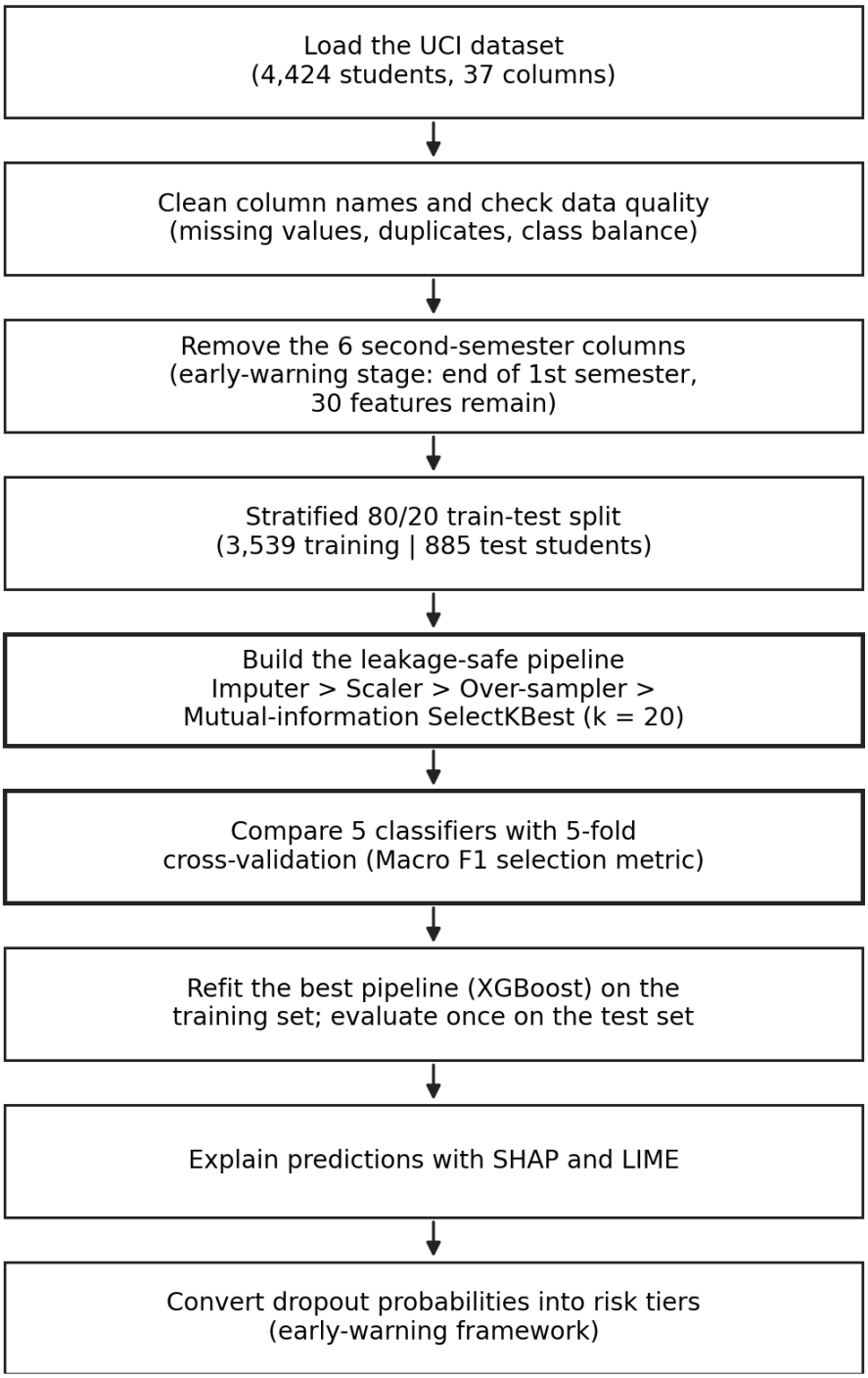
2.6 Evaluation of multi-class classifiers

For multi-class problems with unequal classes, appropriate measures include per-class precision and recall, macro and weighted averages, and balanced accuracy. Generalisations of the area under the ROC curve to more than two classes allow ranking quality to be summarised independently of any decision threshold. This project therefore reports Macro F1 (the model-selection metric), balanced accuracy, weighted F1, per-class recall and one-vs-rest ROC-AUC alongside accuracy.

2.7 Positioning of this work

Rather than proposing a new algorithm, this project integrates well-established components into a single leakage-safe, explainable workflow for an early-warning use case: a clearly defined prediction stage, imbalance handling confined to training folds, in-pipeline feature selection, a transparent model comparison, and individual-level SHAP and LIME explanations.

3. Methodology



Methodology flowchart

3.1 Problem Formulation

Let $D = \{(x_i, y_i)\}$, $i = 1,\ldots,N$, be the student records, where x_i is a vector of $d = 30$ attributes known by the end of the first semester and $y_i \in$ {Dropout, Enrolled, Graduate} is the final academic status. The task is to learn a classifier f that outputs a probability vector $p(x) = (p_{Dropout}, p_{Enrolled}, p_{Graduate})$ and the predicted class $\text{argmax}\ p(x)$. Because the application is early warning, the probability of dropout is as important as the hard label, and the model must be explainable at the level of individual students.

3.2 Dataset

The *Predict Students' Dropout and Academic Success* dataset [1] is obtained from the UCI Machine Learning Repository (dataset 697). It contains **4,424 student records** and 37 columns (36 input attributes plus the *Target* column). The file is semicolon-separated. Data inspection found **0 missing values and 0 duplicate rows**. The attributes fall into six groups (Table 3.1).

Table 3.1: Attribute groups in the dataset

Group	Attributes	Count	Used
Demographic	Marital status, Nacionality, Gender, Age at enrollment, Displaced, Educational special needs, International	7	Yes
Socio-economic	Mother's / Father's qualification, Mother's / Father's occupation, Debtor, Tuition fees up to date, Scholarship holder	7	Yes
Academic at enrollment	Application mode, Application order, Course, Daytime/evening attendance, Previous qualification, Previous qualification (grade), Admission grade	7	Yes
1st-semester performance	Curricular units 1st sem: credited, enrolled, evaluations, approved, grade, without evaluations	6	Yes
Macroeconomic	Unemployment rate, Inflation rate, GDP	3	Yes
2nd-semester performance	Curricular units 2nd sem: credited, enrolled, evaluations, approved, grade, without evaluations	6	**Excluded**

The target distribution is imbalanced: Graduate = 2,209 (49.93%), Dropout = 1,421 (32.12%) and Enrolled = 794 (17.95%). Several attributes (course, application mode, qualifications, occupations) are integer codes of nominal categories; they are used as supplied.

3.3 Early-Warning Prediction Stage and Leakage Control

The prediction point is defined as **after the first semester**. Every column whose name contains “2nd sem” is therefore removed, which drops 6 columns and leaves **30 features**. This design choice deliberately trades some achievable accuracy for realism: a model that used second-semester results would be more accurate but would flag students only after a full year, when intervention is harder. All remaining features are converted to numeric type and any infinite values are replaced by missing values (none were present).

3.4 Train-Test Split

The data are split 80:20 with stratification on the target and a fixed random seed of 42, giving **3,539 training** and **885 test** students. Training class counts are Graduate 1,767, Dropout 1,137, Enrolled 635; test counts are Graduate 442, Dropout 284, Enrolled 159. The test set is used once, after model selection is complete.

3.5 The Modelling Pipeline

All data-dependent steps are chained in one imbalanced-learn pipeline (Figure 3.1), so that during cross-validation each step is fitted on the training fold only and the validation fold is never touched by resampling or selection:

- **Median imputation** (SimpleImputer) – a safeguard for future data; no values are missing in the current file.
- **Standard scaling** (StandardScaler) – zero mean and unit variance, needed by Logistic Regression and SVM and harmless for tree models.
- **Random over-sampling** (RandomOverSampler) – duplicates minority-class rows in the training fold until the classes are balanced. Unlike SMOTE it never creates fractional values for integer-coded categorical attributes. It is applied only when fitting and is skipped at prediction time, so validation and test data are never resampled.
- **Feature selection** (SelectKBest with mutual information) – scores each feature's non-linear dependence on the class and retains the top k = 20 of the 30 features.
- **Classifier** – one of the five models in Table 3.2.

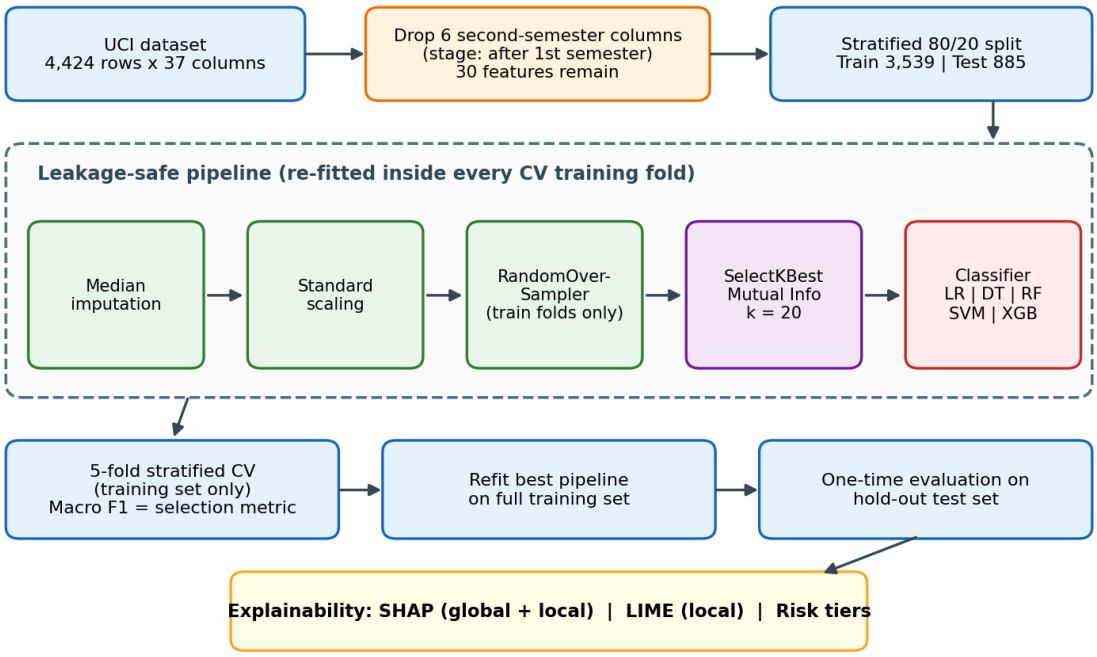


Figure 3.1: Leakage-safe modelling pipeline (re-fitted inside every cross-validation fold)

Table 3.2: Classifiers and hyper-parameters (all with random_state = 42)

Classifier	Configuration
Logistic Regression	max_iter = 3000, default L2 regularisation (multinomial)
Decision Tree	unrestricted depth (max_depth = None), Gini criterion
Random Forest	250 trees
Support Vector Machine	RBF kernel, probability = True
XGBoost	250 trees, max_depth = 4, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8, objective = multi:softprob

3.6 Model Selection Protocol and Metrics

Models are compared with **stratified 5-fold cross-validation on the training set only**. Macro F1 is the primary selection metric, because it weights Dropout, Enrolled and Graduate equally and therefore penalises models that neglect the minority Enrolled class. For class c, precision $P_c = TP_c/(TP_c+FP_c)$, recall $R_c = TP_c/(TP_c+FN_c)$ and $F1_c = 2P_cR_c/(P_c+R_c)$. Macro F1 is the unweighted mean of the three $F1_c$ values; weighted F1 weights them by class support; balanced accuracy is the mean per-class recall. Ranking quality is measured by one-vs-rest ROC-AUC, per class and support-weighted. The best pipeline is refitted on the full training set and evaluated once on the test set.

3.7 Explainability Methods

- **SHAP (permutation explainer).** A model-agnostic permutation explainer is applied to the complete pipeline, so explanations are expressed in terms of the original input features. A background sample of 20 training students is used to represent "absent" features, and 20 randomly selected test students are explained (max_evals = 2d + 1 = 61). The Dropout-class output is plotted as a global beeswarm summary and as a local waterfall plot [7].
- **LIME.** A tabular LIME explainer, trained on the training data with discretised continuous features, explains a single held-out student by fitting a local linear surrogate to perturbed samples around the instance and reporting the ten most influential conditions [8].
- **Model-based importance.** Gain-based feature importance from the fitted XGBoost classifier provides a third, model-internal view of the selected features.

3.8 Experimental Environment and Reproducibility

Experiments were run in Python 3.12 with scikit-learn 1.8.0, XGBoost 3.4.1, SHAP 0.53.0, imbalanced-learn, LIME, pandas 3.0 and seaborn/matplotlib. Two small changes were made to the notebook so that results are exactly repeatable. First, the mutual-information scorer in SelectKBest is wrapped with *random_state = 42*: without a seed, *mutual_info_classif* adds random noise to continuous features, so features near the cut-off can swap between runs and results change slightly. Second, the SHAP permutation explainer is given *seed = 42*. Both changes are marked in the Appendix. Results may still differ slightly under other library versions.

4. Results

4.1 Dataset Exploration

Figure 4.1 shows the class distribution. Graduate is the majority class (49.93%), while Enrolled, the minority class, accounts for only 17.95% of students. After the stratified split the same proportions hold in both partitions, and over-sampling is applied only inside training folds (Section 3.5).

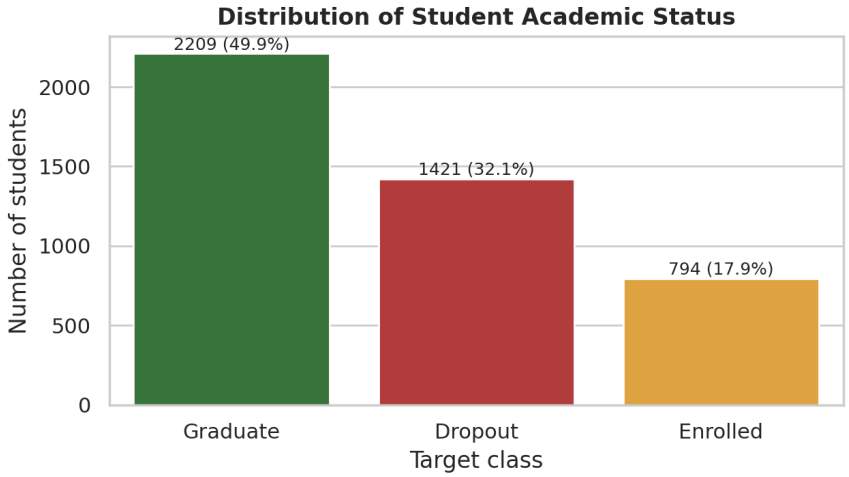


Figure 4.1: Distribution of student academic status in the dataset

4.2 Cross-Validated Model Comparison

Table 4.1 summarises mean scores over the five stratified folds of the training set, ranked by Macro F1. Figure 4.2 visualises the main metrics.

Table 4.1: Five-fold cross-validated performance (training data only)

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1	Macro F1	Balanced Accuracy	Macro F1 Std
XGBoost	0.7372	0.7569	0.7372	0.7437	**0.6941**	0.6988	0.0200
Random Forest	0.7451	0.7366	0.7451	0.7377	0.6764	0.6688	0.0278
Logistic Regression	0.7092	0.7424	0.7092	0.7207	0.6634	0.6684	0.0157
SVM	0.6968	0.7333	0.6968	0.7086	0.6531	0.6578	0.0191
Decision Tree	0.6352	0.6384	0.6352	0.6365	0.5703	0.5703	0.0110

XGBoost obtained the highest Macro F1 (0.6941) and balanced accuracy (0.6988) and was therefore selected automatically. Random Forest achieved a slightly higher plain accuracy (0.7451 vs 0.7372) but a lower Macro F1 (0.6764) and balanced accuracy (0.6688), which indicates better results on the majority classes at the expense of the minority class. The single Decision Tree was clearly the weakest model. Note that the Macro F1 gap between XGBoost and Random Forest (0.0177) is smaller than one fold-to-fold standard deviation (0.020–0.028), so the preference for XGBoost is supported but not conclusive.

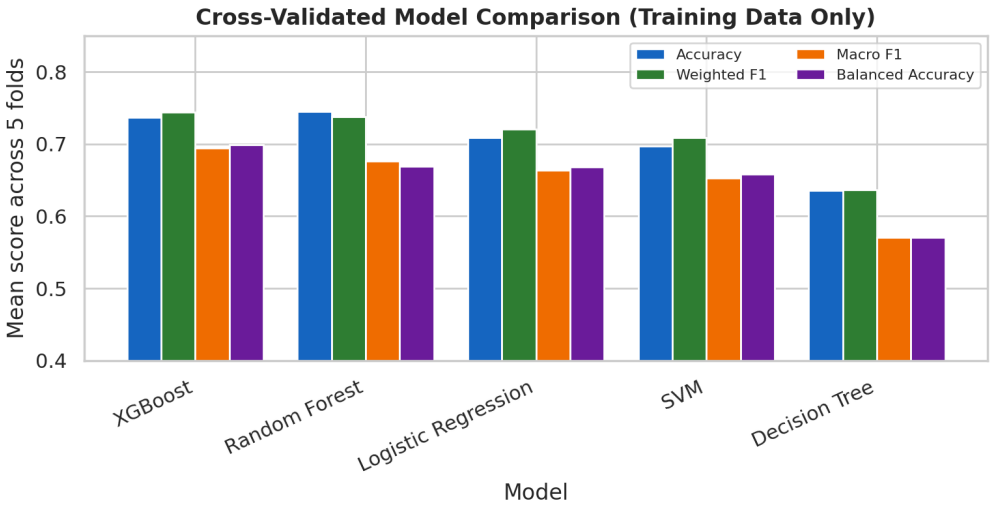


Figure 4.2: Cross-validated comparison of the five classifiers

4.3 Final Evaluation on the Held-Out Test Set

The selected XGBoost pipeline was refitted on all 3,539 training students and evaluated once on the 885 unseen test students (Table 4.2).

Table 4.2: Final test-set metrics of the XGBoost pipeline

Metric	Value
Accuracy	0.7119
Weighted Precision	0.7491
Weighted Recall	0.7119
Weighted F1	0.7241
Macro F1	0.6741
Balanced Accuracy	0.6865
Weighted ROC-AUC (OVR)	0.8783

Table 4.3: Per-class test performance

Class	Precision	Recall	F1-score	Support	ROC-AUC (OvR)
Dropout	0.7824	0.6585	0.7151	284	0.8875
Enrolled	0.4108	0.6226	0.4950	159	0.7738
Graduate	0.8494	0.7783	0.8123	442	0.9100
Macro average	0.6809	0.6865	0.6741	885	–
Weighted average	0.7491	0.7119	0.7241	885	0.8783

Overall accuracy is 71.19% (630 of 885 students correct). Performance differs markedly by class. Graduate students are recognised best (F1 = 0.812, AUC = 0.910) and Dropout students next (F1 = 0.715, AUC = 0.887). The Enrolled class is the weakest (F1 = 0.495, AUC = 0.774): recall is reasonable (0.623) because

over-sampling pushes the model to predict this class more often, but precision is low (0.411), meaning that most students predicted as Enrolled are in fact Graduates or Dropouts.

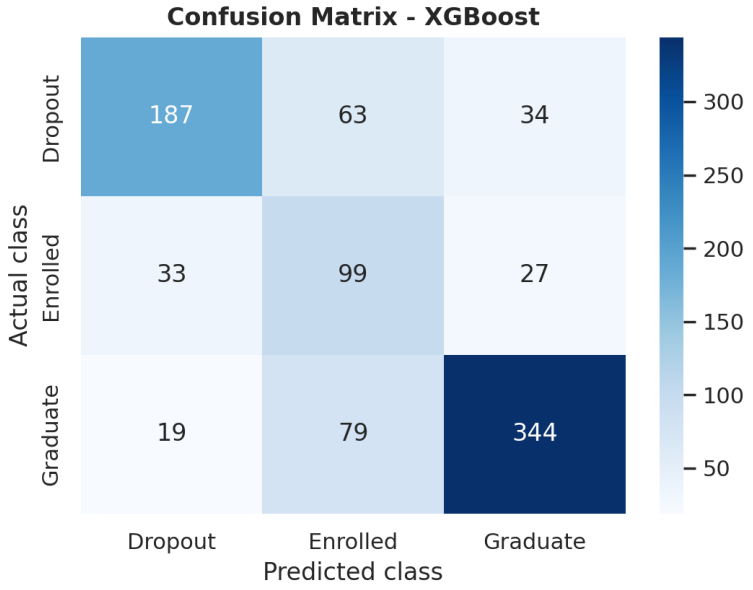


Figure 4.3: Confusion matrix of the XGBoost pipeline on the 885 test students

Table 4.4: Confusion matrix with row-wise recall

Actual \ Predicted	Dropout	Enrolled	Graduate	Total	Recall
Dropout	187	63	34	284	0.658
Enrolled	33	99	27	159	0.623
Graduate	19	79	344	442	0.778

Of the 284 students who actually dropped out, 187 (65.8%) were correctly flagged, 63 were predicted Enrolled and 34 Graduate. The largest single confusion is Graduate predicted as Enrolled (79 students, 17.9% of graduates), reflecting the fact that Enrolled is an intermediate state that overlaps with both outcomes. Notably, few Dropout students are mistaken for Graduates (12.0%), and few Graduates for Dropouts (4.3%): the model rarely makes the most serious errors.

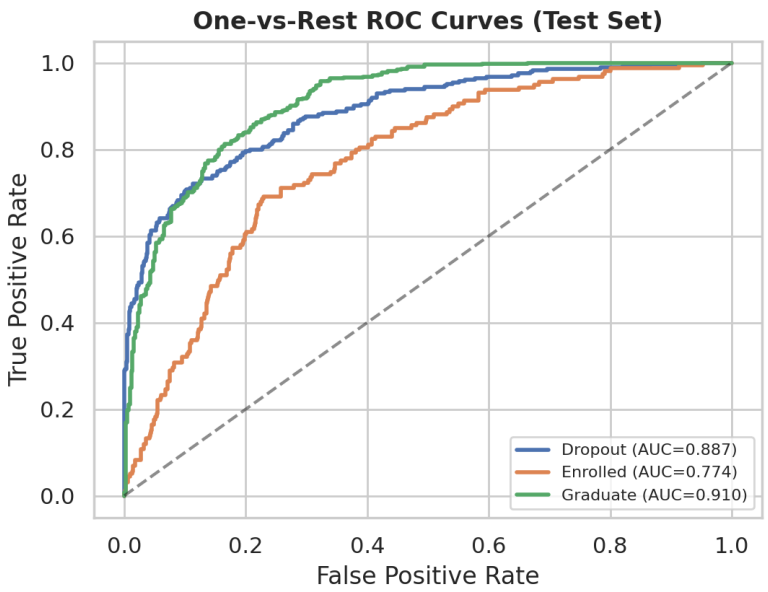


Figure 4.4: One-vs-rest ROC curves on the test set

4.4 Selected Features

Mutual-information feature selection retained 20 of the 30 early-warning features in the final fit (Table 4.5). The two highest-scoring features are first-semester academic indicators (average grade and number of approved units), followed by admission grade. Fifteen of the 20 selected features describe background (academic record at enrollment, socio-economic or demographic information) and five describe first-semester performance, so the selector keeps a mixture of contextual and academic information.

Table 4.5: The 20 features selected by SelectKBest (ranked by mutual-information score on the training set)

Rank	Selected feature	Group	MI score
1	Curricular units 1st sem (grade)	1st-semester performance	0.2314
2	Curricular units 1st sem (approved)	1st-semester performance	0.2148
3	Admission grade	Academic at enrollment	0.1414
4	Curricular units 1st sem (evaluations)	1st-semester performance	0.0882
5	Tuition fees up to date	Socio-economic	0.0846
6	Age at enrollment	Demographic	0.0643
7	Application mode	Academic at enrollment	0.0566
8	Curricular units 1st sem (enrolled)	1st-semester performance	0.0539
9	Course	Academic at enrollment	0.0538
10	Previous qualification (grade)	Academic at enrollment	0.0517
11	Scholarship holder	Socio-economic	0.0481
12	Father's occupation	Socio-economic	0.0475
13	Mother's occupation	Socio-economic	0.0366
14	Previous qualification	Academic at enrollment	0.0293
15	Mother's qualification	Socio-economic	0.0293
16	Debtor	Socio-economic	0.0278
17	Application order	Academic at enrollment	0.0217
18	Curricular units 1st sem (without evaluations)	1st-semester performance	0.0185
19	Father's qualification	Socio-economic	0.0179
20	Displaced	Demographic	0.0155

Features not selected (10): Marital status, Daytime/evening attendance, Nacionality, Educational special needs, Gender, International, Curricular units 1st sem (credited), Unemployment rate, Inflation rate, GDP. Macroeconomic indicators and nationality/international status were not retained, indicating limited marginal association with the outcome in this data.

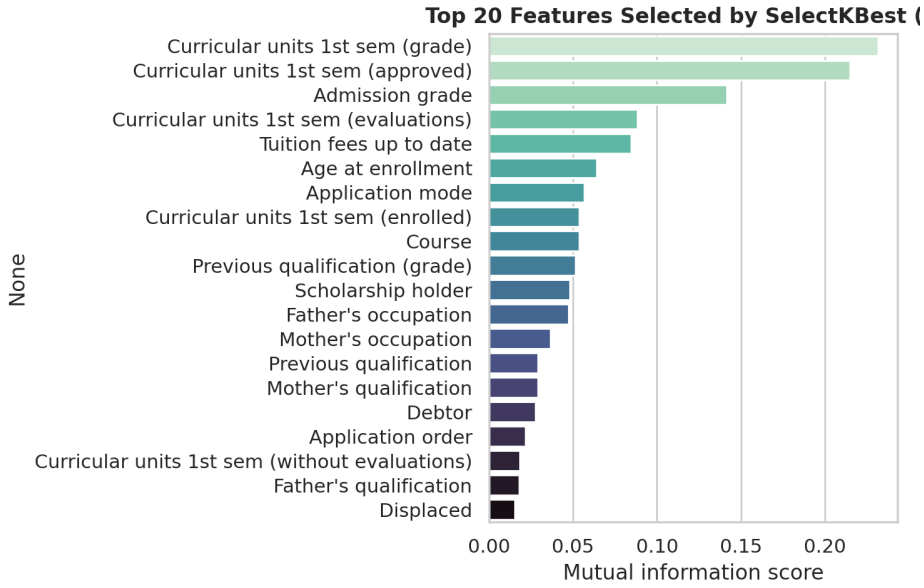


Figure 4.5: Mutual-information scores of the 20 selected features

4.5 Feature Importance

Figure 4.6 shows the fifteen most important features according to the gain-based importance of the fitted XGBoost classifier. The number of curricular units approved in the first semester is clearly the most influential feature, followed by whether tuition fees are up to date, scholarship status, the first-semester average grade and the number of first-semester evaluations. Feature importance indicates association with the model's output, not causation.

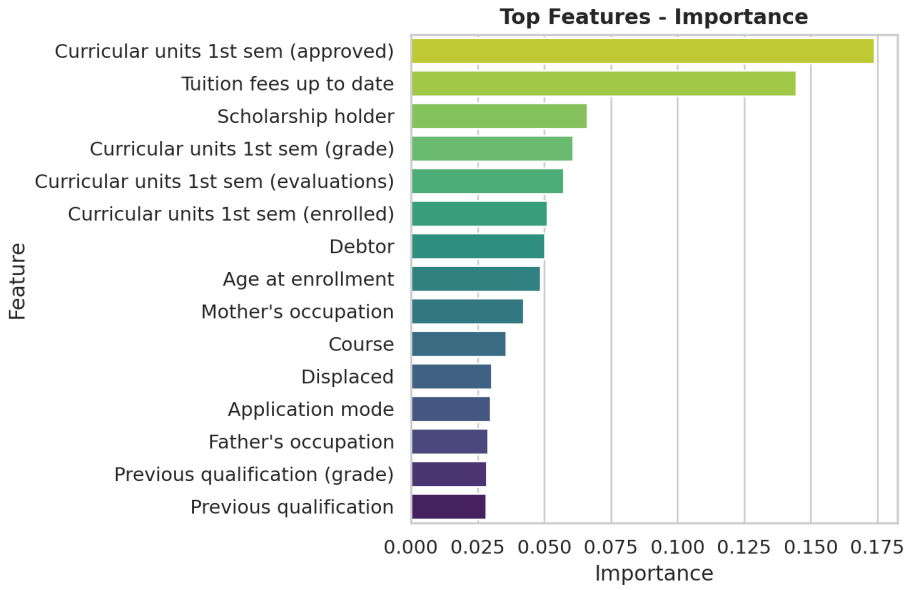


Figure 4.6: Top 15 features by XGBoost gain-based importance

4.6 SHAP Explanations

Global view. Figure 4.7 is the SHAP beeswarm summary for the Dropout class computed over 20 explained test students. Each dot is one student; the horizontal position shows how much the feature raised (right) or lowered (left) that student's predicted probability of dropout, and the colour shows the feature value. Table 4.6 compares the ten largest mean absolute SHAP values with the model's gain importance: the two rankings agree on the leading features.

Table 4.6: Top-10 features by mean |SHAP| for the Dropout class, with XGBoost gain importance

Feature	XGBoost gain importance	Mean |SHAP| (Dropout)
Curricular units 1st sem (approved)	0.1739	0.1673
Tuition fees up to date	0.1446	0.0912
Age at enrollment	0.0487	0.0458
Mother's occupation	0.0422	0.0409
Curricular units 1st sem (enrolled)	0.0512	0.0409
Curricular units 1st sem (grade)	0.0608	0.0386
Curricular units 1st sem (evaluations)	0.0572	0.0289
Course	0.0356	0.0254
Debtor	0.0502	0.0193
Father's occupation	0.0288	0.0189

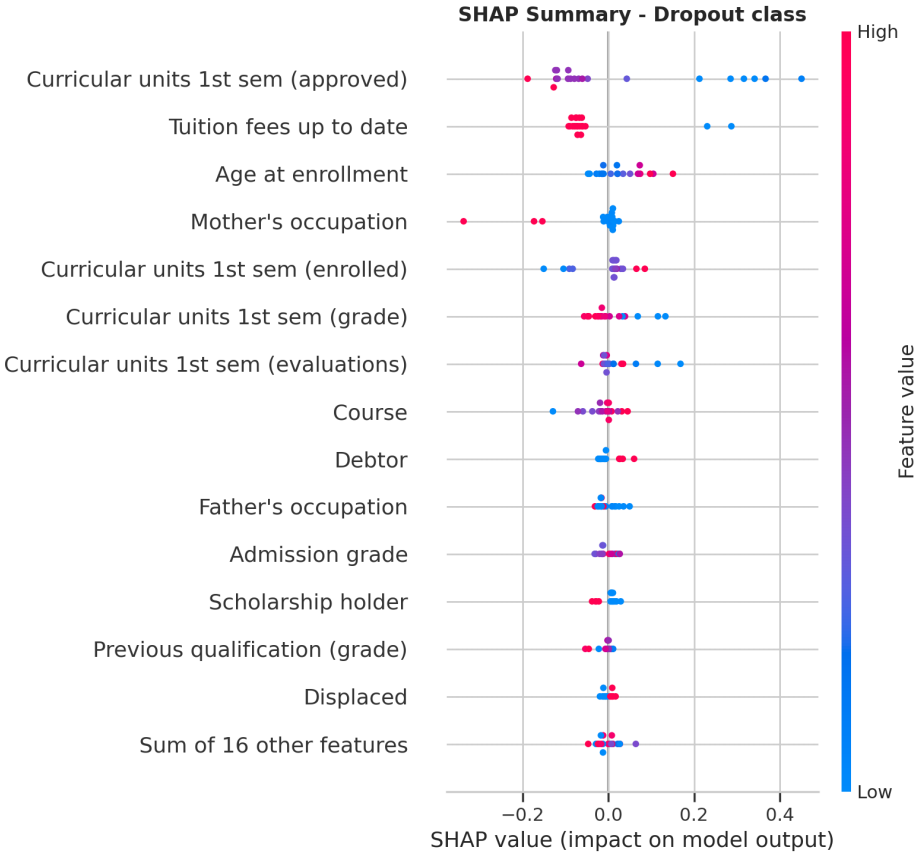


Figure 4.7: SHAP summary (beeswarm) plot for the Dropout class

The pattern of the plot is consistent and intuitive. **Few approved units** in the first semester (blue dots) push the dropout probability up sharply, while many approved units (red dots) lower it. Students whose **tuition fees are not up to date** receive large positive SHAP values, whereas up-to-date students are shifted slightly downwards. **Older age at enrollment** and **debtor status** tend to raise predicted dropout risk, whereas holding a **scholarship** tends to lower it. A higher number of enrolled units combined with low approvals is associated with higher risk, which is consistent with students who register for many units but pass few.

Local view. Figure 4.8 explains a single student from the explained sample (actual status Graduate, predicted Graduate). The base value is the average predicted dropout probability of the background data (0.308) and the final model output for this student is 0.204. The 13 approved units (–0.188) and up-to-date tuition (–0.092) are the strongest protective factors, partly offset by the large number of enrolled units (+0.085) and age at enrollment of 32 years (+0.069).

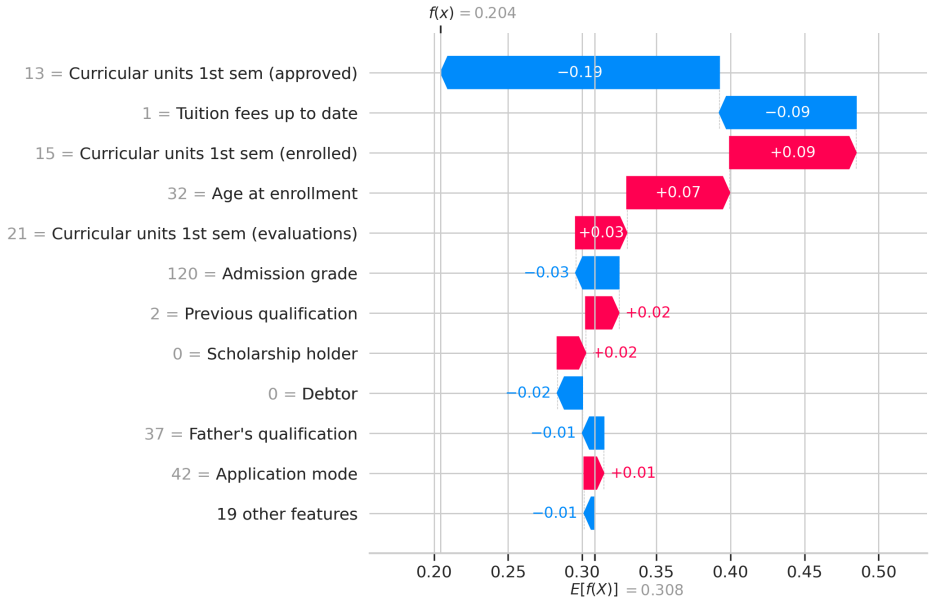


Figure 4.8: SHAP waterfall explanation of one student's dropout probability

4.7 LIME Explanation of an Individual Student

LIME was applied to the first student of the test set (actual status: Graduate; predicted status: Graduate, with class probabilities Dropout 5.31%, Enrolled 7.34%, Graduate 87.35%). Figure 4.9 and Table 4.7 list the ten most influential local conditions for the predicted Graduate class. Green bars support the predicted class and red bars oppose it.

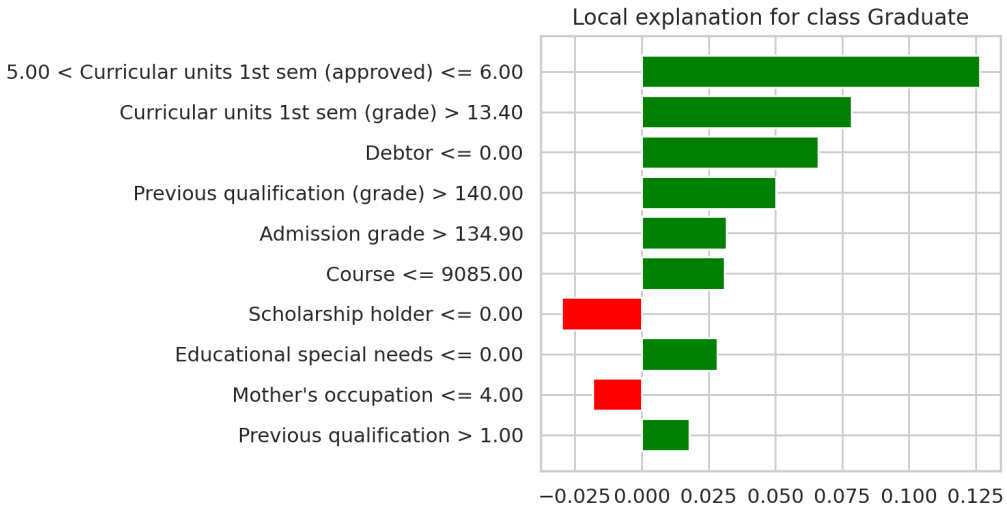


Figure 4.9: LIME local explanation for the predicted class (Graduate)

Table 4.7: LIME local weights for the predicted class (Graduate)

Condition on the student's feature values	Weight
5.00 < Curricular units 1st sem (approved) <= 6.00	+0.1264
Curricular units 1st sem (grade) > 13.40	+0.0785
Debtor <= 0.00	+0.0661
Previous qualification (grade) > 140.00	+0.0501
Admission grade > 134.90	+0.0316
Course <= 9085.00	+0.0309
Scholarship holder <= 0.00	-0.0300
Educational special needs <= 0.00	+0.0283
Mother's occupation <= 4.00	-0.0183
Previous qualification > 1.00	+0.0179

LIME agrees with SHAP on the main message: strong first-semester academic outcomes (approved units, high grade), no debt, and strong prior and admission grades increase the chance of Graduate, whereas the absence of a scholarship slightly reduces it. Because the LIME instance and the SHAP waterfall instance are different students, the two figures should be read as separate illustrations rather than as a like-for-like comparison.

4.8 Individual Prediction and Reusable Prediction Function

The notebook wraps the fitted pipeline in a reusable *predict_student()* function that validates that every required feature is present, preserves the training column order and returns the predicted status with class probabilities. Applied to the first held-out student it returns Graduate with probabilities of 87.35% (Graduate), 7.34% (Enrolled) and 5.31% (Dropout). In a manual-input demonstration, the first record of the dataset was edited to age 20, six enrolled and four approved first-semester units, and tuition fees up to date; the model predicted **Dropout** with probabilities of 77.16% (Dropout), 13.04% (Graduate) and 9.80%

(Enrolled). The full model bundle (pipeline, feature names, label mapping, selected features and prediction-stage metadata) was saved with joblib and reloaded successfully, reproducing the same prediction.

4.9 Early-Warning Framework: Probability Thresholds and Risk Tiers

The project brief asks for an early-warning framework. This section is a **supplementary analysis** built on the same fitted model (its code is in Appendix B) and is not part of the original notebook. Because the model outputs a dropout probability, it can be converted into a flag at any threshold. Table 4.8 shows the trade-off on the test set: lowering the threshold catches more true dropouts (higher recall) but flags more students who would not have dropped out (lower precision). The appropriate operating point is an institutional decision that depends on tutoring capacity.

Table 4.8: Flagging students whose predicted dropout probability is at least a threshold (test set; 284 actual dropouts)

Threshold	Students flagged	% of test set	Precision (flagged who dropped out)	Recall (dropouts caught)
0.20	381	43.1%	61.2%	82.0%
0.30	294	33.2%	70.7%	73.2%
0.40	255	28.8%	77.3%	69.4%
0.50	219	24.7%	83.1%	64.1%
0.60	189	21.4%	87.8%	58.5%
0.70	163	18.4%	89.6%	51.4%

For communication to advisers, three tiers were defined *a priori* (not tuned on the test set): **Low** ($p_{Dropout} <$ 0.30), **Medium** (0.30 to < 0.60) and **High** (≥ 0.60). Table 4.9 and Figure 4.10 show the actual outcomes of test students in each tier.

Table 4.9: Actual outcomes by predicted dropout-risk tier (test set)

Tier	p(Dropout) range	Students	% of test set	Actual Dropout	Actual Enrolled	Actual Graduate
Low	0.00-0.30	591	66.8%	12.9% (76)	17.9%	69.2%
Medium	0.30-0.60	105	11.9%	40.0% (42)	34.3%	25.7%
High	0.60-1.00	189	21.4%	87.8% (166)	9.0%	3.2%

The tiers separate risk well: 87.8% of the 189 High-tier students dropped out, versus 40.0% in the Medium tier and 12.9% in the Low tier. The High tier contains 166 of the 284 actual dropouts (58%) and the Medium tier a further 42. However, 76 actual dropouts (27%) fall in the Low tier, so the system should supplement, not replace, tutors' own judgement. A suggested use is: **High** – prompt personal contact and financial or academic counselling; **Medium** – monitor and offer optional support; **Low** – routine support. The tier cut-offs are illustrative and should be calibrated on institutional data before deployment.

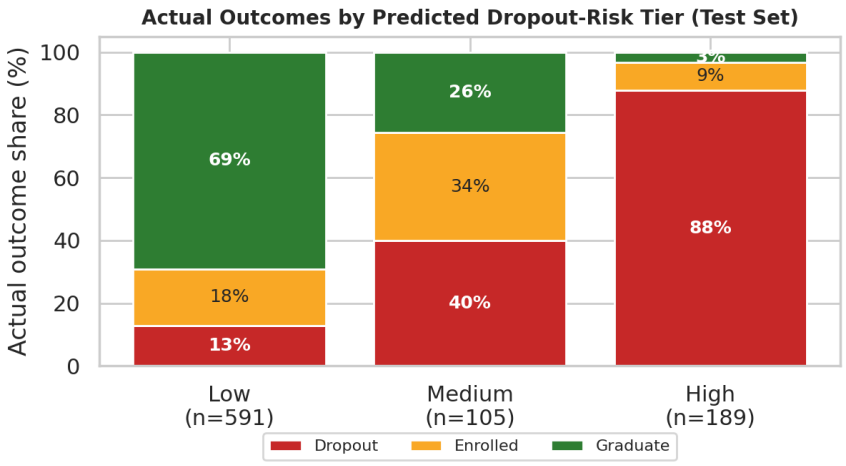


Figure 4.10: Actual outcomes of test students by predicted dropout-risk tier

5. Discussion

5.1 Model Comparison and Selection

Boosted trees offered the best balance across classes. XGBoost led on Macro F1 and balanced accuracy in cross-validation, and Random Forest led on accuracy. Choosing Macro F1 as the selection criterion matters here: it prefers the model that treats the minority Enrolled class better, which is the more relevant behaviour for an early-warning tool. The final test Macro F1 (0.6741) is lower than the cross-validated estimate (0.6941) by about 0.02, roughly one fold-level standard deviation; this is within the range expected from sampling variation on an 885-student test set, and no sign of leakage. Logistic Regression remained competitive (CV Macro F1 0.663), suggesting that a substantial share of the signal is approximately monotonic in a few strong predictors. The unpruned Decision Tree over-fits and is the weakest model.

5.2 Why the Enrolled Class Is Hard

Enrolled students are, by definition, still in progress: their outcome is neither dropout nor graduation yet. Their first-semester profiles overlap with both final outcomes, so there is no clean boundary to learn. Over-sampling raised Enrolled recall to 0.623, but at the price of low precision (0.411) because the model now predicts Enrolled for many students of the neighbouring classes. For early warning this is arguably acceptable, since the safety-critical distinction is between Dropout and the other two outcomes, and direct Dropout/Graduate confusions are comparatively rare (12.0% and 4.3%).

5.3 Risk Factors Identified by the Explanations

Three independent views of the model (gain importance, SHAP and LIME) converge on the same factors. **Academic progress in the first semester**, especially the number of approved units, grade and evaluations, is dominant. **Financial status** is the second pillar: students whose tuition fees are not up to date, or who are flagged as debtors, are at markedly higher modelled risk, while scholarship holders are at lower risk. **Age at enrollment** adds a smaller contribution, with older entrants showing higher risk. These findings are consistent with the view of dropout as a process involving both academic and non-academic integration. They suggest practical levers: early academic tutoring after a poor first semester, and early financial counselling when payment problems appear. They describe associations learned by the model and must not be read as proof of cause.

5.4 Value and Cost of the Early-Warning Design

Excluding second-semester data means the model can act after one semester rather than two. This report did not measure how much accuracy that restriction costs, because no run with second-semester features was performed; such an ablation would quantify the trade-off and is listed as future work. What can be said is that the model, even restricted to first-semester information, separates risk groups usefully (Table 4.9).

5.5 Limitations

- **Single split and no confidence intervals.** Test metrics come from one stratified split of 885 students; with 159 Enrolled students the class-level estimates in particular are uncertain. Repeated cross-validation or bootstrapping would give interval estimates.
- **Coded categorical variables.** Course, application mode, qualifications and occupations are integer codes that are scaled and used as if numeric. Models can still exploit them (trees split on thresholds), but the numerical order has no real meaning; one-hot or target encoding could be explored.
- **Selection after over-sampling.** In the pipeline, mutual-information scores are computed after random over-sampling, so duplicated minority rows can influence the nearest-neighbour estimates. Moving the selector before the sampler is a simple alternative to test.
- **Limited tuning.** Hyper-parameters were fixed rather than tuned, and the choice k = 20 was not optimised.
- **Small SHAP sample.** SHAP values were computed for 20 test students against a 20-student background to keep run-time low. The qualitative pattern is clear, but the numerical mean |SHAP| values would be more stable with a larger sample.
- **Probability calibration.** Over-sampling shifts predicted probabilities, so the probability tiers are rank-based indicators rather than calibrated risks.
- **Single-institution data and fairness.** All data come from one Portuguese institution, so transfer to other settings is unproven. Attributes such as gender, age and parental occupation are present; using them in decisions that affect students raises fairness concerns and should be audited before deployment.

5.6 Practical Implications

The prototype is lightweight (a single saved pipeline of 20 selected inputs), explainable at student level, and produces probabilities that can be turned into risk tiers. Used as a decision-support tool, for instance to order a tutor's contact list after the first semester, it can help direct limited support resources to students who most need them. It should always be used with human oversight, explanations should accompany each flag, and interventions should be supportive rather than punitive.

6. Conclusion

6.1 Summary of Findings

This project built and evaluated an explainable early-warning model for student dropout using the UCI *Predict Students' Dropout and Academic Success* dataset. By defining the prediction point as the end of the first semester and embedding imputation, scaling, over-sampling and feature selection in a single cross-validated pipeline, the study avoids both temporal and methodological leakage. Five classifiers were compared by cross-validated Macro F1, and XGBoost was selected and evaluated once on a held-out test set. SHAP, LIME and gain-based importance consistently identified first-semester academic progress and financial standing as the strongest risk signals.

Table 6.1: Project scorecard

Evaluation item	Result
Dataset	4,424 students; 36 attributes; 3 classes (Graduate / Dropout / Enrolled)
Prediction stage	After first semester (6 second-semester columns removed; 30 features retained)
Imbalance handling	RandomOverSampler inside training folds only
Feature selection	Mutual information, SelectKBest, k = 20 of 30
Models compared	Logistic Regression, Decision Tree, Random Forest, SVM, XGBoost
Best model (CV Macro F1)	XGBoost (0.6941 ± 0.0200)
Test accuracy	71.19%
Test Macro F1	0.6741
Test balanced accuracy	0.6865
Test weighted ROC-AUC (OvR)	0.8783
Per-class F1 (Dropout / Enrolled / Graduate)	0.715 / 0.495 / 0.812
Top risk factors	First-semester approved units, tuition fees up to date, age at enrollment, first-semester enrolled units and grade, debtor status
High-risk tier outcome	87.8% of High-tier students dropped out (Low tier: 12.9%)

6.2 Recommendations for Practitioners

- Define the prediction time point explicitly and remove all information from after it; a model that sees the future is not an early-warning model.
- Do all fitting steps (scaling, over-sampling, feature selection) inside cross-validation, and seed every stochastic component so that results can be reproduced.
- Select models on a class-balanced metric such as Macro F1 and report per-class recall, not only accuracy.
- Present predictions as probabilities with an explanation, and agree risk-tier cut-offs with the advisers who will act on them.
- Audit the model for fairness and calibrate it on the institution's own data before any deployment.

6.3 Future Work

- Run an ablation with and without second-semester features to quantify the accuracy/earliness trade-off, and build a two-stage system that updates risk after each semester.
- Tune hyper-parameters and the number of selected features with nested cross-validation; report confidence intervals from repeated cross-validation.
- Test one-hot or target encoding for categorical-coded attributes and moving feature selection before over-sampling; compare SMOTE-NC and class-weighting alternatives.
- Calibrate predicted probabilities (for example, isotonic regression) and choose risk-tier thresholds with cost-sensitive analysis.
- Compute SHAP values on a larger sample using the exact tree explainer, and add stability and fairness analyses across gender, age and course.
- Treat the outcome as ordinal (Dropout < Enrolled < Graduate) or add the time dimension through survival models, which may better capture the intermediate Enrolled state.

References

[1] V. Realinho, J. Machado, L. Baptista, and M. V. Martins, “Predicting student dropout and academic success,” *Data*, vol. 7, no. 11, p. 146, 2022.

[2] I. Guyon and A. Elisseeff, “An introduction to variable and feature selection,” *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[3] C. Ambroise and G. J. McLachlan, “Selection bias in gene extraction on the basis of microarray gene-expression data,” *Proceedings of the National Academy of Sciences*, vol. 99, no. 10, pp. 6562–6566, 2002.

[4] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[5] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[7] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

Appendix A: Source Code Implementation

The code below is the complete notebook *Student_Dropout_Risk_Prediction_Revised.ipynb*, listed block by block. The only two modifications to the original cells (seeding the mutual-information scorer and the SHAP explainer, plus the one import this needs) are marked with the comment `[REPRODUCIBILITY PATCH]` (see Section 3.8). Long lines are wrapped for printing.

Block 0 — Install dependencies

```
# (Colab) install dependencies
%pip -q install imbalanced-learn xgboost shap lime
```

Block 1 — Import libraries and define labels

```
import warnings
warnings.filterwarnings("default")

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.pipeline import Pipeline as SklearnPipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    balanced_accuracy_score, classification_report,
    confusion_matrix, roc_auc_score, make_scorer
)
from sklearn.inspection import permutation_importance

from functools import partial  # [REPRODUCIBILITY PATCH] needed for seeded MI
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import RandomOverSampler
from xgboost import XGBClassifier

RANDOM_STATE = 42
CLASS_NAMES = ["Dropout", "Enrolled", "Graduate"]
TARGET_MAPPING = {"Dropout": 0, "Enrolled": 1, "Graduate": 2}
LABEL_MAPPING = {v: k for k, v in TARGET_MAPPING.items()}

print("Libraries imported successfully.")
```

Block 2 — Load the dataset

```
# Google Colab: upload the UCI dataset CSV.
# The expected file is usually named "data.csv" and uses semicolons as separators.
try:
    from google.colab import files
    uploaded = files.upload()
    if not uploaded:
        raise ValueError("No file was uploaded.")
    csv_path = next(iter(uploaded.keys()))
except ImportError:
    # For Jupyter/local execution, place the CSV beside this notebook or update this path.
    csv_path = "data.csv"

if "csv_path" not in globals():
    csv_path = "data.csv"

try:
    df = pd.read_csv(csv_path, sep=";")
    # Fallback for a comma-separated file if semicolon parsing produced one column.
    if df.shape[1] == 1:
        df = pd.read_csv(csv_path)
except FileNotFoundError as exc:
    raise FileNotFoundError(
        "Dataset not found. Upload the UCI dataset CSV in Colab or place it at data.csv."
    ) from exc
# Remove tabs/spaces accidentally included in column names.
```

```
df.columns = df.columns.astype(str).str.replace(chr(9), "", regex=False).str.strip()

if "Target" not in df.columns:
    raise ValueError(
        "The dataset must contain a 'Target' column. Check the CSV file and separator."
    )

print("Dataset loaded:", df.shape)
display(df.head())
```

Block 3 — Inspect the dataset

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
display(df.dtypes.to_frame("Data type"))

print("\nMissing values per column:")
display(df.isna().sum().sort_values(ascending=False).to_frame("Missing values").head(15))

print("\nDuplicate rows:", int(df.duplicated().sum()))
print("\nTarget values:")
print(df["Target"].value_counts(dropna=False))
```

Block 4 — Visualize target distribution

```
target_counts = df["Target"].value_counts(dropna=False)

plt.figure(figsize=(7, 4))
sns.barplot(x=target_counts.index.astype(str), y=target_counts.values)
plt.title("Distribution of Student Academic Status")
plt.xlabel("Target class")
plt.ylabel("Number of students")
plt.tight_layout()
plt.show()

print("Class percentages:")
display((df["Target"].value_counts(normalize=True) * 100).round(2).to_frame("Percentage"))
```

Block 5 — Define the early-warning prediction stage and prepare features

```
# Define the prediction point: after the first semester.
# To avoid using future information, remove second-semester performance features.
if df["Target"].isna().any():
    df = df.dropna(subset=["Target"]).copy()

X_all = df.drop(columns=["Target"]).copy()
y_raw = df["Target"].astype(str).str.strip()

future_columns = [
    col for col in X_all.columns
    if "2nd sem" in col.lower() or "second semester" in col.lower()
]
X = X_all.drop(columns=future_columns).copy()

# Convert features to numeric and replace infinite values with missing values.
X = X.apply(pd.to_numeric, errors="coerce")
X = X.replace([np.inf, -np.inf], np.nan)

unknown_targets = sorted(set(y_raw.unique()) - set(TARGET_MAPPING))
if unknown_targets:
    raise ValueError(f"Unexpected target labels found: {unknown_targets}")

y = y_raw.map(TARGET_MAPPING).astype(int)

print("Prediction stage: after the first semester")
print("Excluded future-information columns:", future_columns if future_columns else "None found")
print("Features retained:", X.shape[1])
print("X shape:", X.shape, "| y shape:", y.shape)
print("Target mapping:", TARGET_MAPPING)
```

Block 6 — Check data quality

```
# Inspect whether any feature is completely missing.
all_missing = X.columns[X.isna().all()].tolist()
if all_missing:
    print("Dropping completely empty feature columns:", all_missing)
    X = X.drop(columns=all_missing)
print("Missing values remaining before imputation:", int(X.isna().sum().sum()))
```

```
print("Duplicate records in the feature table:", int(X.duplicated().sum()))

# Keep all features numeric; median imputation will be fitted on training folds only.
print("Final feature count:", X.shape[1])
display(X.head())
```

Block 7 — Create the train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)

print("Training rows:", len(X_train))
print("Test rows:", len(X_test))
print("\nTraining class counts:")
display(y_train.map(LABEL_MAPPING).value_counts().to_frame("Count"))
print("\nTest class counts:")
display(y_test.map(LABEL_MAPPING).value_counts().to_frame("Count"))
```

Block 8 — Inspect class imbalance and explain the balancing strategy

```
print("Training class distribution before balancing:")
display(y_train.map(LABEL_MAPPING).value_counts().to_frame("Count"))

print("\nBalancing strategy:")
print(
    "RandomOverSampler will duplicate minority-class training rows. "
    "Unlike ordinary SMOTE, it will not create fractional values for encoded categorical features."
)
print("The sampler will run inside each cross-validation training fold and will never touch the test set.")
```

Block 9 — Set the number of selected features

```
k_features = min(20, X_train.shape[1])
print(f"SelectKBest will retain {k_features} features in each training fold.")
print("Feature selection is part of the model pipeline to reduce leakage during cross-validation.")
```

Block 10 — Define classifiers and preprocessing pipelines

```
models = {
    "Logistic Regression": LogisticRegression(
        max_iter=3000, class_weight=None, random_state=RANDOM_STATE
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=None, random_state=RANDOM_STATE
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=250, random_state=RANDOM_STATE, n_jobs=1
    ),
    "SVM": SVC(
        probability=True, random_state=RANDOM_STATE
    ),
    "XGBoost": XGBClassifier(
        n_estimators=250,
        max_depth=4,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="multi:softprob",
        num_class=3,
        eval_metric="mlogloss",
        random_state=RANDOM_STATE,
        n_jobs=1
    )
}

def make_model_pipeline(estimator):
    return Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("sampler", RandomOverSampler(random_state=RANDOM_STATE)),
        # [REPRODUCIBILITY PATCH] seeded mutual information
        ("selector", SelectKBest(score_func=partial(mutual_info_classif, random_state=RANDOM_STATE),
            k=k_features)),
        ("classifier", estimator)
    ])

pipelines = {name: make_model_pipeline(model) for name, model in models.items()}
```

```
print("Pipelines prepared:", ", ".join(pipelines.keys()))
```

Block 11 — Compare models using cross-validation

```
# Model selection uses cross-validation on the training set only.
# The held-out test set remains untouched until the final evaluation.
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {
    "accuracy": "accuracy",
    "precision_weighted": "precision_weighted",
    "recall_weighted": "recall_weighted",
    "f1_weighted": "f1_weighted",
    "f1_macro": "f1_macro",
    "balanced_accuracy": "balanced_accuracy"
}

cv_rows = []
for name, pipeline in pipelines.items():
    print(f"Cross-validating: {name}")
    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=1,
        error_score="raise",
        return_train_score=False
    )
    cv_rows.append({
        "Model": name,
        "Accuracy": scores["test_accuracy"].mean(),
        "Weighted Precision": scores["test_precision_weighted"].mean(),
        "Weighted Recall": scores["test_recall_weighted"].mean(),
        "Weighted F1": scores["test_f1_weighted"].mean(),
        "Macro F1": scores["test_f1_macro"].mean(),
        "Balanced Accuracy": scores["test_balanced_accuracy"].mean(),
        "CV Macro F1 Std": scores["test_f1_macro"].std()
    })

cv_results = pd.DataFrame(cv_rows).sort_values("Macro F1", ascending=False).reset_index(drop=True)
display(cv_results.round(4))
```

Block 12 — Visualize cross-validation results

```
plt.figure(figsize=(12, 6))
cv_results.set_index("Model")[["Accuracy", "Weighted F1", "Macro F1", "Balanced Accuracy"]].plot(
    kind="bar", figsize=(12, 6)
)
plt.title("Cross-Validated Model Comparison (Training Data Only)")
plt.ylabel("Mean score across folds")
plt.ylim(0, 1)
plt.xticks(rotation=35, ha="right")
plt.tight_layout()
plt.show()

print("Primary model-selection metric: Macro F1.")
print("Macro F1 gives equal weight to Dropout, Enrolled, and Graduate.")
```

Block 13 — Select and fit the best model

```
best_model_name = cv_results.loc[cv_results["Macro F1"].idxmax(), "Model"]
best_model = pipelines[best_model_name]

print("Selected model based on cross-validated Macro F1:", best_model_name)
best_model.fit(X_train, y_train)
print("Final pipeline fitted on the full training set.")
```

Block 14 — Evaluate on the untouched test set

```
# Final evaluation: the test set is used only after model selection is complete.
y_pred = best_model.predict(X_test)
y_probability = best_model.predict_proba(X_test)

final_metrics = {
    "Accuracy": accuracy_score(y_test, y_pred),
    "Weighted Precision": precision_score(y_test, y_pred, average="weighted", zero_division=0),
    "Weighted Recall": recall_score(y_test, y_pred, average="weighted", zero_division=0),
    "Weighted F1": f1_score(y_test, y_pred, average="weighted", zero_division=0),
    "Macro F1": f1_score(y_test, y_pred, average="macro", zero_division=0),
    "Balanced Accuracy": balanced_accuracy_score(y_test, y_pred)
}
```

```
try:
    final_metrics["Weighted ROC-AUC (OVR)"] = roc_auc_score(
        y_test, y_probability, labels=[0, 1, 2], multi_class="ovr", average="weighted"
    )
except ValueError as exc:
    print("ROC-AUC could not be calculated:", exc)

print("Final held-out test metrics:")
display(pd.DataFrame([final_metrics]).round(4))
```

Block 15 — Plot the confusion matrix

```
cm = confusion_matrix(y_test, y_pred, labels=[0, 1, 2])

plt.figure(figsize=(7, 5))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES
)
plt.xlabel("Predicted class")
plt.ylabel("Actual class")
plt.title(f"Confusion Matrix — {best_model_name}")
plt.tight_layout()
plt.show()
```

Block 16 — Print the classification report

```
print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1, 2],
    target_names=CLASS_NAMES,
    digits=4,
    zero_division=0
))

per_class_recall = recall_score(y_test, y_pred, labels=[0, 1, 2], average=None, zero_division=0)
print("\nClass-wise recall:")
for class_name, score in zip(CLASS_NAMES, per_class_recall):
    print(f"{class_name}: {score:.3f}")

print("\nInterpretation reminder: inspect Enrolled-class recall carefully; overall accuracy can hide weak
    performance on a minority class.")
```

Block 17 — Calculate per-class ROC-AUC

```
# Per-class ROC-AUC (one-vs-rest), when each class is present in the test set.
from sklearn.preprocessing import label_binarize

y_test_binary = label_binarize(y_test, classes=[0, 1, 2])
per_class_auc = {}
for idx, class_name in enumerate(CLASS_NAMES):
    try:
        per_class_auc[class_name] = roc_auc_score(y_test_binary[:, idx], y_probability[:, idx])
    except ValueError:
        per_class_auc[class_name] = np.nan

display(pd.DataFrame({
    "Class": list(per_class_auc.keys()),
    "ROC-AUC (OVR)": list(per_class_auc.values())
}).round(4))

print("ROC-AUC measures class separation; it is not the same as accuracy.")
```

Block 18 — Display selected features and feature importance

```
# Extract the final selected feature names from the fitted pipeline.
selector = best_model.named_steps["selector"]
selected_features = X_train.columns[selector.get_support()].tolist()
classifier = best_model.named_steps["classifier"]

print("Selected features used by the final model:")
for feature in selected_features:
    print("-", feature)

if hasattr(classifier, "feature_importances_"):
    importances = classifier.feature_importances_
    importance_df = pd.DataFrame({"Feature": selected_features, "Importance": importances})
else:
    # For linear/SVM models, use permutation importance on the held-out set for interpretation.
    perm = permutation_importance(
        best_model, X_test, y_test, scoring="f1_macro",
```

```
        n_repeats=3, random_state=RANDOM_STATE, n_jobs=1
    )
    importance_df = pd.DataFrame({
        "Feature": X_test.columns,
        "Importance": perm.importances_mean
    })

importance_df = importance_df.sort_values("Importance", ascending=False)
display(importance_df.head(15).round(4))

plt.figure(figsize=(10, 7))
sns.barplot(data=importance_df.head(15), x="Importance", y="Feature")
plt.title("Top Features — Model-Based or Permutation Importance")
plt.tight_layout()
plt.show()

print("Feature importance indicates model association, not causation.")
```

Block 19 — Calculate SHAP values

```
# SHAP explanations using the complete fitted pipeline and raw input features.
# Permutation SHAP is model-agnostic and works with this preprocessing pipeline.
import shap

feature_names = X_train.columns.tolist()
background = X_train.sample(n=min(20, len(X_train)), random_state=RANDOM_STATE)
shap_rows = X_test.sample(n=min(20, len(X_test)), random_state=RANDOM_STATE)

def predict_proba_for_shap(data):
    data_frame = pd.DataFrame(data, columns=feature_names)
    return best_model.predict_proba(data_frame)

shap_explainer = shap.Explainer(
    predict_proba_for_shap,
    background,
    algorithm="permutation",
    seed=RANDOM_STATE  # [REPRODUCIBILITY PATCH] seeded SHAP
)
max_evals = 2 * len(feature_names) + 1
shap_values = shap_explainer(shap_rows, max_evals=max_evals)

print("SHAP values calculated.")
print("SHAP value array shape:", np.asarray(shap_values.values).shape)
```

Block 20 — Plot global and individual SHAP explanations

```
# Global SHAP plot and one individual explanation for the Dropout class.
shap_array = np.asarray(shap_values.values)
base_array = np.asarray(shap_values.base_values)
class_index = TARGET_MAPPING["Dropout"]

if shap_array.ndim == 3:
    class_shap_values = shap_array[:, :, class_index]
    if base_array.ndim == 2:
        class_base_values = base_array[:, class_index]
    else:
        class_base_values = base_array
elif shap_array.ndim == 2:
    class_shap_values = shap_array
    class_base_values = base_array
else:
    raise ValueError(f"Unexpected SHAP value shape: {shap_array.shape}")

dropout_explanation = shap.Explanation(
    values=class_shap_values,
    base_values=class_base_values,
    data=shap_rows.to_numpy(),
    feature_names=feature_names
)

print("Global SHAP summary for the Dropout class:")
shap.plots.beeswarm(dropout_explanation, max_display=15)

print("Individual SHAP explanation for the first explained student:")
shap.plots.waterfall(dropout_explanation[0], max_display=12)
```

Block 21 — Generate an individual LIME explanation

```
# LIME explanation for one individual student.
from lime.lime_tabular import LimeTabularExplainer

lime_explainer = LimeTabularExplainer(
```

```
    training_data=X_train.to_numpy(dtype=float),
    feature_names=feature_names,
    class_names=CLASS_NAMES,
    mode="classification",
    discretize_continuous=True,
    random_state=RANDOM_STATE
)

def predict_proba_for_lime(data):
    data_frame = pd.DataFrame(data, columns=feature_names)
    return best_model.predict_proba(data_frame)

lime_index = 0
lime_row = X_test.iloc[lime_index].to_numpy(dtype=float)
lime_exp = lime_explainer.explain_instance(
    lime_row,
    predict_proba_for_lime,
    labels=[0, 1, 2],
    num_features=min(10, len(feature_names))
)

print("Actual status:", LABEL_MAPPING[int(y_test.iloc[lime_index])])
print("Predicted status:", LABEL_MAPPING[int(y_pred[lime_index])])
lime_exp.show_in_notebook(show_table=True)
fig = lime_exp.as_pyplot_figure(label=int(y_pred[lime_index]))
plt.tight_layout()
plt.show()
```

Block 22 — Demonstrate prediction on a held-out student

```
# Test an individual held-out student and show all class probabilities.
student_index = 0
student_row = X_test.iloc[[student_index]]
predicted_code = int(best_model.predict(student_row)[0])
probabilities = best_model.predict_proba(student_row)[0]

print("Actual status:", LABEL_MAPPING[int(y_test.iloc[student_index])])
print("Predicted status:", LABEL_MAPPING[predicted_code])
print("\nPredicted class probabilities:")
for class_code, probability in enumerate(probabilities):
    print(f"{LABEL_MAPPING[class_code]}: {probability * 100:.2f}%")
```

Block 23 — Create a reusable student prediction function

```
def predict_student(student_data):
    """Predict a student's status from a dict or pandas Series of eligible features."""
    student_df = pd.DataFrame([dict(student_data)])

    # Require every training feature and keep the same column order as training.
    missing = [col for col in feature_names if col not in student_df.columns]
    extra = [col for col in student_df.columns if col not in feature_names]
    if missing:
        raise ValueError(f"Missing required feature(s): {missing}")
    if extra:
        print("Ignoring extra feature(s):", extra)

    student_df = student_df.reindex(columns=feature_names)
    student_df = student_df.apply(pd.to_numeric, errors="coerce")

    prediction_code = int(best_model.predict(student_df)[0])
    probabilities = best_model.predict_proba(student_df)[0]

    result = {
        "predicted_status": LABEL_MAPPING[prediction_code],
        "probabilities": {
            LABEL_MAPPING[i]: round(float(probabilities[i]) * 100, 2)
            for i in range(len(CLASS_NAMES))
        }
    }

    print("\nSTUDENT STATUS PREDICTION")
    print("Predicted status:", result["predicted_status"])
    print("Class probabilities:")
    for label, probability in result["probabilities"].items():
        print(f"  {label}: {probability:.2f}%")

    return result

print("predict_student() is ready. Required feature count:", len(feature_names))
```

Block 24 — Test the prediction function

```
# Demonstrate the prediction function with one real row from the test set.
# This is a demonstration example, not a guarantee of future student outcomes.
sample_student = X_test.iloc[0].to_dict()
sample_result = predict_student(sample_student)
```

Block 25 — Save the complete model bundle

```
# Save a complete bundle so preprocessing and feature order stay consistent.
import joblib

model_bundle = {
    "model": best_model,
    "model_name": best_model_name,
    "feature_names": feature_names,
    "target_mapping": TARGET_MAPPING,
    "label_mapping": LABEL_MAPPING,
    "prediction_stage": "After the first semester",
    "excluded_future_features": future_columns,
    "selected_features": selected_features
}

joblib.dump(model_bundle, "student_dropout_model_bundle.joblib")
print("Saved: student_dropout_model_bundle.joblib")
```

Block 26 — Reload the bundle and verify predictions

```
# Reload the saved bundle and verify that predictions still work.
loaded_bundle = joblib.load("student_dropout_model_bundle.joblib")
loaded_model = loaded_bundle["model"]
loaded_features = loaded_bundle["feature_names"]

test_input = X_test.iloc[[0]].reindex(columns=loaded_features)
loaded_prediction = int(loaded_model.predict(test_input)[0])
loaded_probabilities = loaded_model.predict_proba(test_input)[0]

print("Reloaded model:", loaded_bundle["model_name"])
print("Prediction after reload:", loaded_bundle["label_mapping"][loaded_prediction])
print("Probabilities after reload:")
for idx, probability in enumerate(loaded_probabilities):
    print(f"{loaded_bundle['label_mapping'][idx]}: {probability * 100:.2f}%")
```

Block 27 — Try a manual student-input example

```
# Manual demonstration input: start from a valid record so every required feature exists.
# Replace values below with valid values from the dataset's coding scheme.
manual_student_data = X.iloc[0].to_dict()

# Example edits (change or remove these as appropriate for your test case).
if "Age at enrollment" in manual_student_data:
    manual_student_data["Age at enrollment"] = 20
if "Curricular units 1st sem (approved)" in manual_student_data:
    manual_student_data["Curricular units 1st sem (approved)"] = 4
if "Curricular units 1st sem (enrolled)" in manual_student_data:
    manual_student_data["Curricular units 1st sem (enrolled)"] = 6
if "Tuition fees up to date" in manual_student_data:
    manual_student_data["Tuition fees up to date"] = 1

print("Manual input features:", len(manual_student_data))
manual_result = predict_student(manual_student_data)
```

Block 28 — Run final sanity checks

```
# Final sanity checks for the report.
print("Dataset rows used:", len(df))
print("Early-warning features:", X.shape[1])
print("Excluded later-semester features:", len(future_columns))
print("Selected model:", best_model_name)
print("Final test Macro F1:", round(final_metrics["Macro F1"], 4))
print("Final test Balanced Accuracy:", round(final_metrics["Balanced Accuracy"], 4))
print("\nBefore submission: restart the runtime and run all cells from top to bottom.")
```

Appendix B: Supplementary Code for the Risk-Tier Analysis (Section 4.9)

This snippet is not part of the original notebook. It continues from the fitted `best_model`, `X_test` and `y_test` objects defined in Blocks 13 and 14.

```
# Supplementary: probability-threshold operating points and risk tiers
p_dropout = best_model.predict_proba(X_test)[:, 0]       # column 0 = Dropout
is_dropout = (y_test.values == 0)

print("Threshold | flagged | precision | recall")
for t in [0.20, 0.30, 0.40, 0.50, 0.60, 0.70]:
    flag = p_dropout >= t
    precision = is_dropout[flag].mean() if flag.sum() else float("nan")
    recall = flag[is_dropout].mean()
    print(f"{t:.2f} | {flag.sum():4d} | {precision*100:5.1f}% | {recall*100:5.1f}%")

# Risk tiers fixed a priori (not tuned on the test set)
tiers = [("Low", 0.00, 0.30), ("Medium", 0.30, 0.60), ("High", 0.60, 1.01)]
for name, lo, hi in tiers:
    mask = (p_dropout >= lo) & (p_dropout < hi)
    outcome = y_test.values[mask]
    print(f"{name:6s} n={mask.sum():4d}  Dropout={np.mean(outcome==0)*100:5.1f}%  "
          f"Enrolled={np.mean(outcome==1)*100:5.1f}%  Graduate={np.mean(outcome==2)*100:5.1f}%")
```